HushBro

Android Studio Build Guide

Droidrush 2026 · Team INVICTUS · Problem 04: The Ward of Silence

Purpose: a practical, beginner-friendly implementation plan for building the offline notification manager in Android Studio using Kotlin and Jetpack Compose. Work through one milestone at a time, build and test after every milestone, and keep the project compiling.

App name: **HushBro**

Package ID: **com.invictus.hushbro**

Prepared from the Droidrush problem statement and the supplied Hushward guide, with the project name and Android Studio workflow updated.

How to use this guide

This guide turns the submitted problem statement into an ordered implementation plan. Do not ask an AI coding assistant to build the entire app in one prompt. Complete one step, compile, test the stated behaviour, commit it, and then move on.

Recommended workflow

- Create the project in Android Studio and make the first empty build succeed.
- Save this PDF in your repository. Create BUILD_GUIDE.md (optional, for copy/paste prompts) and PROGRESS.md.
- Implement one numbered step at a time. Give the assistant the Master Context and that step only.
- After every step: Sync Gradle, Build > Make Project, run on an emulator/device, test the acceptance checks, and commit.
- If a step fails, stop adding features. Fix only the current failure using the exact error log and relevant files.
- Before changing dependencies, confirm the versions are compatible with the installed Android Gradle Plugin, Kotlin, and KSP.

Master Context — paste into every coding-assistant chat

```
You are helping build HushBro for Droidrush 2026, Team INVICTUS.
Use the supplied HushBro Build Guide and PROGRESS.md as the source of truth.
Implement only the requested step. Do not rewrite unrelated files.

Platform: Android Studio, Kotlin, single app module, minSdk 26,
Jetpack Compose + Material 3, Navigation Compose, Room, Coroutines/Flow,
WorkManager, DataStore, Paging 3, Hilt, SQLCipher for Android, Vico charts.
Application ID/package: com.invictus.hushbro.

NON-NEGOTIABLE:
1. Fully offline. Do not add android.permission.INTERNET or networking libraries.
2. NotificationListenerService is read-only observation. Never reply, invoke
   another app's RemoteInput, or use its action PendingIntents.
3. Redact sensitive text BEFORE database insertion or logging.
4. Listener callbacks only extract minimal fields and enqueue; no database,
   regex, rule evaluation, or heavy work in the callback.
5. The first matching enabled rule wins.
6. Store only redacted content. Never log raw notification text in release code.
7. Keep the app compiling. Add/update tests and PROGRESS.md at the end.
8. Do not invent APIs. Check the installed library version's actual API.
9. Implement exactly one guide step and report files changed, build/test results,
   manual checks, known limitations, and the next action.
```

Step prompt template

```
Read PROGRESS.md and the HushBro Build Guide. Implement STEP [N] only.
Follow the Do this and Done when sections. Inspect existing code before editing.
Do not refactor completed steps. Run the relevant tests and assembleDebug.
If blocked, explain the smallest blocker and do not fake a successful build.
Update PROGRESS.md with completed work and the exact next action.
```

Important implementation interpretation

The problem statement calls the app offline and privacy-first. That means no backend and no INTERNET permission. Android still requires the user to explicitly grant Notification Access; Android 13+ also requires POST_NOTIFICATIONS for HushBro's own digest/reminder notifications. Notification routing means an internal label/bucket in HushBro's history—not moving another app's notification to a different Android system channel.

1. What you are building

HushBro observes notifications on the device, redacts sensitive content, applies user-defined rules, classifies notifications, stores safe history locally, and optionally bundles low-priority items into a digest. Users can search history, snooze an item, and view local usage insights.

Area	Expected behaviour
Onboarding & permissions	Explain access plainly; open Notification Access settings; re-check permission when returning.
Capture	Read package, app label, title, text, timestamp, category, key and visibility from posted notifications.
Privacy pipeline	Hide private-app content and mask OTP/card/account-like values before persistence.
Rules	Ordered, deterministic rules: MUTE, MARK_IMPORTANT, ROUTE; first match wins.
Classification	Tag each item OTP, PROMO, SOCIAL or OTHER on-device.
History	Searchable, app-grouped timeline; mark read, clear app/all, status and importance indicators.
Digest & quiet hours	Hold eligible low-priority alerts and post one HushBro digest on schedule.
Snooze	Re-post a stored, already-redacted reminder at the selected time.
Insights & export	Noisiest apps, hourly rate, totals and redacted CSV export.
Reliability	Deduplication, burst queue, batch writer, pruning, listener rebind and recovery.
Security	Room encrypted with SQLCipher; key protected by Android Keystore; no network permission.

Suggested implementation order

Finish Steps 1–11 first. This yields the essential notification manager and UI. Then implement digest, snooze, insights, CSV and encryption. SQLCipher is deliberately late because switching an existing development database to encrypted storage requires clearing/reinstalling the app during development.

2. Before opening Android Studio

- Install a current stable Android Studio version and let its Setup Wizard install the Android SDK.
- Use JDK 17 or the JDK bundled with the Android Studio/Gradle setup recommended by the project template. Do not change Gradle JDK casually.
- Have an Android emulator with API 33+ for notification permission testing, plus ideally a physical Android phone for Notification Access and OEM battery behaviour.
- Create a Git repository before major changes. Commit each verified milestone.

Create the project in Android Studio

- Welcome screen → New Project.
- Choose Empty Activity (the Compose template). Avoid the Views-only template.
- Name: HushBro.
- Package name: com.invictus.hushbro.
- Language: Kotlin.
- Minimum SDK: API 26 (Android 8.0).
- Choose a save location, finish, and wait for Gradle sync.
- Run the untouched starter app once before changing dependencies.

Android Studio template labels can vary by version. If the template asks for a build configuration language, choose Kotlin DSL if available (build.gradle.kts). Keep a single app module.

3. Project structure

Use this package layout. Create packages with right-click app/src/main/java/com/invictus/hushbro → New → Package. Android Studio may show the path in compact form; that is normal.

```
com.invictus.hushbro
  HushBroApp.kt
  MainActivity.kt
  di/
  data/
    db/       entities, DAOs, database, converters
    repo/     notification, rule, stats repositories
    prefs/    DataStore settings
  service/    HushBroListenerService, ListenerBridge
  pipeline/   RawNotification, NotificationPipeline,
              Redactor, RuleEngine, Classifier
  work/       DigestWorker, SnoozeWorker, PruneWorker, Scheduler
  notify/     Channels, DigestNotifier, SnoozeNotifier
  ui/
    theme/
    nav/
    onboarding/
    history/
    rules/
    settings/
    insights/
  util/       KeystoreHelper, CsvExporter, TimeUtils
```

Core data model

Model	Fields / intent
NotificationEntity	id, key, packageName, appLabel, title?, text?, postTime, tag, status, isRead, isImportant, channelLabel?, matchedRuleId?, textHash. Unique dedupe index plus app/time and time indexes.
RuleEntity	id, priority (lower runs first), optional appPackage, keyword, action, optional channelLabel, enabled.
AppPrefEntity	packageName, isPrivate, digestEnabled, optional quiet-hour overrides.
Settings (DataStore)	onboardingDone, digest interval, digest tags, global quiet hours, retention days.
RawNotification	Temporary in-memory object only; raw title/text must not be persisted or logged before redaction.

Status/tag vocabulary

Tags: OTP, PROMO, SOCIAL, OTHER. **Statuses:** KEPT, MUTED, HELD, DIGESTED, SNOOZED. Keep these as enums with Room type converters; do not scatter string literals through the app.

Dependency strategy

Use stable versions only, declared centrally in gradle/libs.versions.toml where practical. Add dependencies in small groups and sync after each group. Hilt/Kotlin/KSP/AGP compatibility is version-sensitive, so do not copy arbitrary versions from old tutorials. Use the official AndroidX, Hilt, SQLCipher and Vico documentation for the exact version you install.

- Foundation: Compose Material 3, Navigation Compose.
- Storage: Room runtime + KTX + Paging integration, Room compiler via KSP, DataStore Preferences.
- Async/background: Coroutines, WorkManager, Hilt Work integration.
- Dependency injection: Hilt Android, Hilt compiler/plugin, AndroidX Hilt navigation/work as needed.
- Later: Paging Compose, SQLCipher for Android + AndroidX SQLite, Vico Compose Material 3, JUnit/Truth/Room testing.

> Avoid adding every library at once if Gradle sync fails. First build the Compose app, then add a compatible group, sync, and build again.

STEP 1 — Android Studio project and app shell

Goal: a clean HushBro Compose app that installs and opens.

Do this

- Confirm applicationId and namespace are com.invictus.hushbro in app/build.gradle.kts.
- Set minSdk = 26. Leave compileSdk/targetSdk at the stable versions supported by the current template unless the event rules specify otherwise.
- Add dependencies in controlled groups, syncing and building after each group.
- Create HushBroApp.kt and register the Application class only after Hilt is configured. Annotate MainActivity with @AndroidEntryPoint when using Hilt.
- Set android:allowBackup="false". Do not add INTERNET permission.
- Create package folders and a simple Home screen displaying HushBro.

Verify

- Build > Make Project succeeds.
- Run on emulator; app opens without crashing.
- Search app/src/main/AndroidManifest.xml and all source files for INTERNET; no permission or network code should exist.

If Hilt causes plugin/compiler errors, temporarily complete the Compose shell first, then add Hilt using one consistent compatible version set. Do not keep changing unrelated Gradle versions to silence errors.

STEP 2 — Room database (initially unencrypted)

Goal: define entities and duplicate-safe DAO operations before capturing real notifications.

- Create NotificationEntity, RuleEntity and AppPrefEntity with the fields listed in Section 3. Add enum converters.
- Create NotificationDao: batch insert with OnConflictStrategy.IGNORE; paged search ordered newest first; app summaries; per-app paging; mark read/all read; clear all/per app; update status; held-for-digest; delete older than timestamp.
- Create RuleDao: observe ordered by priority, upsert, delete and priority swap. Create AppPrefDao: observe all, upsert, get one app preference.
- Create WardDatabase/HushBroDatabase with DAO getters, version 1 and schema export enabled.
- Use destructive migration fallback only during development. Do not ship destructive migration behaviour as a substitute for migrations.

Verify

- Instrumented test inserts the same logical notification twice and confirms one row.
- Search query returns only matching title/text/app label rows.
- DAO ordering is newest first; clearing data and marking read behave correctly.

STEP 3 — Repositories, dependency injection and channels

- Add a Hilt module that provides the database and DAOs as singletons.
- Create NotificationRepository, RuleRepository and StatsRepository. Keep SQL inside DAOs; repositories expose Flow/Pager and write methods.
- Create SettingsStore with DataStore Preferences. Expose a Flow and setter for every setting; define safe defaults.
- Create two notification channels owned by HushBro: digest (low importance) and snooze (default importance). Create them at app startup.
- Provide an application-scoped CoroutineScope using SupervisorJob + Dispatchers.Default if the pipeline needs a long-lived consumer.

Verify

- App starts with Hilt and no dependency injection crash.
- Both channels appear under Android Settings → Apps → HushBro → Notifications.

STEP 4 — Notification access and listener service

- Create HushBroListenerService extending NotificationListenerService and declare it in the manifest with BIND_NOTIFICATION_LISTENER_SERVICE and the exact service intent action.
- On onboarding, explain what is read (app, title/text, time, category) and what is never done (no network, no replies). Button opens Settings.ACTION_NOTIFICATION_LISTENER_SETTINGS.
- When returning to the app, re-check listener access and show a clear banner if disabled.
- In onNotificationPosted, ignore HushBro's own notifications, group summaries and ongoing notifications if consistent with the product requirements.
- Extract only package, key, title, text (BIG_TEXT then TEXT fallback), post time, category and visibility. For this step, log only safe test metadata; do not log raw real notification text.
- Handle unknown app labels safely with package name fallback. Add a package visibility declaration for launcher apps if app labels are needed on Android 11+.
- onListenerDisconnected should request rebind. onListenerConnected should later back-fill active notifications.

Verify

- Grant access in system settings, post a test notification and confirm the listener receives it.
- HushBro's own digest/reminder notifications are ignored by capture.

Notification Access is a special user-granted setting, not a normal runtime permission dialog. The app cannot silently enable it for the user.

STEP 5 — Buffered pipeline and batch writer

This step protects responsiveness during notification bursts. The listener callback must remain tiny.

- Define RawNotification and interfaces for Redactor, RuleEngine and Classifier. Temporary pass-through implementations are acceptable until their steps.
- Create a bounded Kotlin Channel with capacity 512 and DROP_OLDEST overflow policy. enqueue() uses trySend and never suspends.
- Start one consumer coroutine on Dispatchers.Default. Receive one item, then drain up to 100 items or 250 ms, whichever comes first.
- Process each item off the callback thread, then call one repository insertBatch for the batch. Ensure batch insertion uses a transaction.
- Use a ListenerBridge to let the pipeline ask the connected service to cancel a notification by key when a MUTE rule applies.
- On listener reconnect, enqueue getActiveNotifications() results; dedupe prevents duplicate history rows.
- Create a stable textHash from the same redacted title/text fields on every processing attempt. Use a unique index that includes key, postTime and hash, as specified in the model.

Verify

- Burst of 50 test notifications creates expected rows without UI jank or ANR.
- Reconnecting does not duplicate existing notifications.
- Confirm callback contains no Room query, regex work, rule evaluation or synchronous file operation.

STEP 6 — Redaction and private apps

- Implement OTP masking for 4–8 digit codes near terms such as OTP, code, verification, PIN and password.
- Mask 13–19 digit card/account-like sequences (spaces/dashes allowed), retaining only the final four digits.
- If visibility is VISIBILITY_SECRET, replace sensitive text with [Hidden]. For apps marked private, replace both title and text with [Hidden], keeping package/time.
- Keep an in-memory StateFlow snapshot of app privacy preferences, refreshed from the DAO. Never query the database for each notification.
- Precompile regex patterns once. Redact before insertion, rule matching, CSV export or any persistent logs.
- The classifier may inspect raw text transiently for OTP tagging, but raw text must not be stored or logged.

Verify

- Unit test: “Your OTP is 482913” stores “Your OTP is ****”.
- Unit test: card-like number exposes only its final four digits.
- Unit test: private app stores [Hidden] for title/text.
- Inspect test database and logs; no raw OTP appears.

STEP 7 — Deterministic rule engine

- Maintain an in-memory, precompiled snapshot of enabled rules sorted by ascending priority.
- A rule matches when its optional app package matches and its keyword appears case-insensitively in title + space + text.
- Evaluate rules in order; return the first match only. If no rule matches, return no action.
- Actions: MUTE cancels the original notification and stores MUTED status; MARK_IMPORTANT sets isImportant; ROUTE stores a channelLabel/internal bucket.
- Match only against redacted title/text so rules and metadata cannot accidentally persist sensitive content.
- When rules change, update the snapshot from Flow immediately; new notifications should not require app restart.

Verify

- Unit tests cover first-match wins, app-specific rules, disabled rules and no-match.
- Manual test: MUTE rule removes a test notification from the shade while a redacted history row remains.

ROUTE is an internal HushBro label. Android does not let HushBro move another app's notification into a different system notification channel.

STEP 8 — On-device classifier

- Tag as OTP first when the raw text matches the OTP pattern.
- Otherwise map Android social/message categories to SOCIAL and promo category to PROMO.
- Otherwise check local keyword lists: promo (sale, offer, discount, coupon, cashback, deal, % off); social (liked, commented, followed, tagged, mentioned, friend request).
- If nothing matches, use OTHER. Keep lists in one Kotlin object.
- Run classification locally before raw text is discarded; persist only the tag and redacted fields.

Verify

- Unit tests cover every tag and precedence: a notification mentioning OTP and a sale is tagged OTP.

STEP 9 — History timeline

- Add Navigation Compose destinations: History, Rules, Insights and Settings. Use a bottom navigation bar.
- History top section groups by app label with total/unread counts. Tapping an app expands its paged history; while searching, show a flat result list.
- Use a search field with 300 ms debounce and Paging 3. Render only the current page; do not load all history into memory.
- Each row shows title, two lines of text, relative time, tag, important star and status badge. Unread rows are bold.
- Actions: tap marks read; per-app menu offers mark all read and clear app; top action offers clear all with confirmation.
- Show empty state and an access-disabled banner.

Verify

- Search, mark-read, clear-app and clear-all work and survive restart.
- Generate thousands of test rows and scroll smoothly.

STEP 10 — Rule editor UI

- List rules in priority order with enabled switch, edit/delete and up/down priority controls.
- Add/edit form: app picker (Any app plus apps observed in history), keyword, action dropdown, and channel label only for ROUTE.
- Reject blank keywords and invalid ROUTE rules without a label.
- Display “First matching rule wins” near the list.

Verify

- A newly created rule affects the next notification immediately.
- Reordering two matching rules changes the winning action as expected.

STEP 11 — Settings and app preferences

- Privacy: list observed apps and a private toggle.
- Digest: interval (hourly/daily), tags included (PROMO/SOCIAL by default as chosen by product), and per-app digest toggle.
- Quiet hours: global start/end and optional per-app overrides. Handle overnight windows correctly (for example 22:00–07:00).
- Retention: 7, 30 or 90 days. About/Privacy explains local storage, no network, no replies and how to clear all data.
- Persist global settings in DataStore and per-app settings in AppPrefDao. Refresh pipeline snapshots live.
- Changing digest interval calls Scheduler.rescheduleDigest(); it can be a stub until Step 12.

Verify

- Settings survive process restart.
- Marking an app private causes its next notification to store [Hidden].

At this point, the core Pass-tier build should be demonstrable. Commit and tag this milestone before moving on.

STEP 12 — Digest and quiet hours

- Define low priority as matching selected digest tags or an app-level digest preference, excluding important items and OTP.
- Eligible items are marked HELD and cancelled from the shade. Never hold OTP or important items.
- Use WorkManager PeriodicWorkRequest for hourly or daily digest. Periodic work has a platform minimum interval (typically 15 minutes); it is not an exact alarm.
- Use a unique work name such as digest and ExistingPeriodicWorkPolicy.UPDATE. Reschedule when settings change.
- Outside quiet hours, load held items and post one notification on the digest channel with total count and app counts; mark included rows DIGESTED.
- Inside global quiet hours, skip posting. Check per-app quiet hours during classification/holding. Respect POST_NOTIFICATIONS on Android 13+.
- Request POST_NOTIFICATIONS in context when the user enables digest/reminders, and explain why it is needed.

Verify

- Force a worker run with held demo items and confirm exactly one digest notification.
- Confirm no digest appears during quiet hours and held items appear in the next digest.

WorkManager is reliable background scheduling, not a guarantee of exact wall-clock delivery. Device power policy can delay jobs.

STEP 13 — Snooze until

- Add snooze actions: 15 minutes, 1 hour, this evening (18:00), tomorrow morning (09:00), and custom time.
- Mark item SNOOZED; cancel its active notification if present; enqueue a unique OneTimeWorkRequest with an initial delay.
- Worker posts a HushBro-owned reminder using only stored, already-redacted content. Cap active snoozes (for example 50) to avoid accidental spam.
- Tap should open HushBro history or the source app's launcher activity if available; never invoke notification action PendingIntents or RemoteInput.
- After posting the reminder, update status according to the UI contract and prevent duplicate re-posts.

Verify

- Use a short test delay (about one minute); reminder appears once.
- SNOOZED status is visible and then clears/updates after re-post.
- Reboot test: WorkManager work remains scheduled.

STEP 14 — Insights and charts

- Add DAO Flow queries: top apps over a time range, hourly buckets, totals, muted, held and important.
- Fill missing hourly buckets with zero so chart time axes remain continuous.
- Insights range selector: 24 hours, 7 days, 30 days. Show summary cards, noisiest-app chart and hourly-rate chart.
- Use Vico Compose Material 3. Follow the README for the exact installed Vico 2.x API; chart APIs differ across major versions.
- Handle empty history with a friendly empty state.

Verify

- Charts update when new notifications arrive.

- Range filters change counts and chart points correctly.

STEP 15 — Redacted CSV export

- Use ActivityResultContracts.CreateDocument("text/csv") so the user chooses the destination; do not request broad storage permissions.
- Stream rows in chunks (for example 500) on Dispatchers.IO. Escape quotes, commas and line breaks according to CSV rules.
- Columns: time_iso, app, title, text, tag, status, important, rule_id.
- Export only fields already redacted in the database. Add an Export CSV button on Insights and show success/failure feedback.

Verify

- Open exported file in Excel/Sheets; commas and quotes remain in the right cells.
- Export 10,000 rows without freezing the UI.

STEP 16 — SQLCipher encrypted database

Do this only after the earlier features work. The supplied problem statement explicitly calls for SQLCipher-backed Room storage.

- Create KeystoreHelper: generate an AES-GCM key in AndroidKeyStore once. Generate a random 32-byte database passphrase, encrypt it with the Keystore key, and store ciphertext + IV in app-private DataStore/preferences.
- On launch, decrypt the passphrase. Never store it in plaintext. Avoid logging it.
- Load SQLCipher's native library once, then configure Room with SupportOpenHelperFactory and the passphrase bytes using the API for the installed SQLCipher version.
- Zero temporary passphrase byte arrays when practical. Do not change DAOs/entities while making this storage change.
- Clear app data/uninstall and reinstall the development build before first encrypted run; the old unencrypted DB cannot simply be opened as encrypted.

Verify

- Database file no longer begins with the plain SQLite header “SQLite format 3”.
- Search strings from a test notification are not readable in the database file.
- All previous features pass again after a clean install.

SQLCipher and Room APIs are version-sensitive. Follow Zetetic's current Room integration instructions rather than guessing a class or import.

STEP 17 — Reliability, pruning and privacy audit

- Create daily PruneWorker that deletes records older than retentionDays. Run VACUUM only when safe and outside an active transaction; verify whether your database setup supports the operation as written.
- Add a battery optimisation help row and manufacturer-specific tips, but do not promise that the OS will never stop the service.
- Check listener rebind logic and test access toggled off/on. Back-fill active notifications after reconnect.
- Add data-extraction rules excluding private database content from cloud/device transfer, in addition to allowBackup=false.
- Audit manifest and source for INTERNET, networking clients, RemoteInput, sendReply and use of other apps' action PendingIntents.
- Privacy screen lists exactly what is collected, where it is stored, how redaction works, what never happens, and how to clear data.

Verify

- Force-stop/reboot and verify service recovery after the user has granted access; behaviour can vary by OEM.
- Run an audit of the merged manifest, not only the source manifest.
- Confirm no network permission or networking dependency is present.

STEP 18 — Tests, demo and submission

- Unit tests: Redactor, RuleEngine, Classifier, CSV escaping, quiet-hour boundary cases and dedupe hash stability.
- Instrumented tests: Room duplicate insert and paging/search.
- Demo categories: OTP, promo, social and normal notification; show mute, important, route, digest, snooze, history, insight and CSV.
- Test a burst of 500 notifications; record row count, UI responsiveness and chart updates.
- Create debug/release APK as required by the event, write README with setup, permission explanation, architecture/data-flow diagram and screenshots.
- Install on a second device/emulator and walk through onboarding from a clean state.

Demo commands

These Android shell commands may vary by emulator/Android version. If a command is unavailable, create a tiny local test sender app or use Android Studio's notification testing method. Do not use real OTPs or private messages in demo recordings.

```
adb shell cmd notification post -S bigtext -t "Bank" otp1 "Your OTP is 482913. Do not share."
adb shell cmd notification post -S bigtext -t "ShopX" promo1 "50% off sale ends tonight, use coupon FLASH"
adb shell cmd notification post -S bigtext -t "Social" soc1 "A friend liked your photo"

# Burst test (run in a shell that supports this loop):
for i in $(seq 1 500); do
  adb shell cmd notification post -t "Chat" burst$i "message $i"
done
```

4. Android Studio testing checklist

Test	How to verify
Build	Build > Make Project; then Run app. Resolve all errors before next step.
Logcat	Filter by package com.invictus.hushbro. Never print raw notification text or secrets.
Manifest	Open merged manifest view and confirm no INTERNET permission.
Notification Access	Settings → Apps/Special app access → Notification access → enable HushBro; return and check banner disappears.
Own notification permission	On Android 13+, deny and enable POST_NOTIFICATIONS to test both paths.
Database	App Inspection → Database Inspector; inspect only test content and confirm redaction.
WorkManager	Use App Inspection/Background Task tools or a test-only worker trigger; periodic work is not exact.
Clean install	Uninstall or clear data before verifying first-run onboarding and SQLCipher setup.
Burst	Post many synthetic notifications and watch Logcat, database count and UI responsiveness.

Troubleshooting

Symptom	Likely fix
Listener sees nothing	Check Notification Access was granted, service manifest permission/action are exact, and access wasn't revoked after reinstall.
App labels are package IDs	Check package visibility for launcher apps and fallback handling.
Duplicate history rows	Verify unique index, key/postTime, and stable hash derived from redacted fields.
UI freezes on burst	Move DB/regex/rule work out of onNotificationPosted; use bounded queue and batch transaction.
Digest missing	Check permission, held rows, quiet hours, unique WorkManager scheduling and periodic-work delay.
Snooze posts twice	Use unique work names and ensure worker status/idempotency checks are correct.
SQLCipher startup crash	Check native library load, passphrase retrieval, SQLCipher/Room compatibility and clear old unencrypted dev DB.
Hilt/KSP build errors	Align AGP, Kotlin, KSP and Hilt plugin/compiler versions; sync and rebuild without mixing alpha/stable releases.
Vico API unresolved	Use the README for the exact installed Vico version; do not paste code written for another major version.

5. PROGRESS.md template

Create a plain text file named PROGRESS.md at the project root. Update it at the end of every step so a new coding session can continue safely.

```
# HushBro progress

Last completed step: 0
Currently in progress: Step 1 — project setup
Next action: finish Step 1 verification

## Done
- (step number: result and key files)

## Decisions / deviations
- (what changed from this guide and why)

## Known bugs / TODO
- (exact error or remaining manual check)

## Last verified
- Gradle sync:
- assembleDebug:
- Tests:
- Device/emulator:
```

6. Suggested Git milestones

```
git init
git add .
git commit -m "step 1: create HushBro Compose project"

# After each verified step:
git add .
git commit -m "step N: short description"

# Example milestone tags:
git tag pass-tier-complete
git tag victory-features-complete
git tag submission-ready
```

7. Final submission checklist

- ■ App name, launcher label and README consistently say HushBro.
- ■ Application ID/package is com.invictus.hushbro; no stale Hushward package names remain.
- ■ No INTERNET permission in source or merged manifest; no networking libraries.
- ■ Onboarding explains Notification Access and POST_NOTIFICATIONS separately.
- ■ Sensitive text is redacted before database writes; private apps and secret visibility are hidden.
- ■ Rule ordering is deterministic and first-match wins.
- ■ History search, mark-read, clear operations and paging work.
- ■ Digest and snooze use HushBro-owned notification channels and never invoke replies/actions from other apps.
- ■ SQLCipher encryption is verified on disk; key material is protected with Android Keystore.
- ■ Retention/pruning, CSV export, charts, burst test and recovery checks have been run.
- ■ README, architecture diagram, screenshots and APK are ready.
- ■ Fresh-install test on a second emulator/device passes.

Implementation principle: a smaller, verified app is better than a large codebase that does not compile. Complete Steps 1–11 first, then add optional victory-tier features in order.